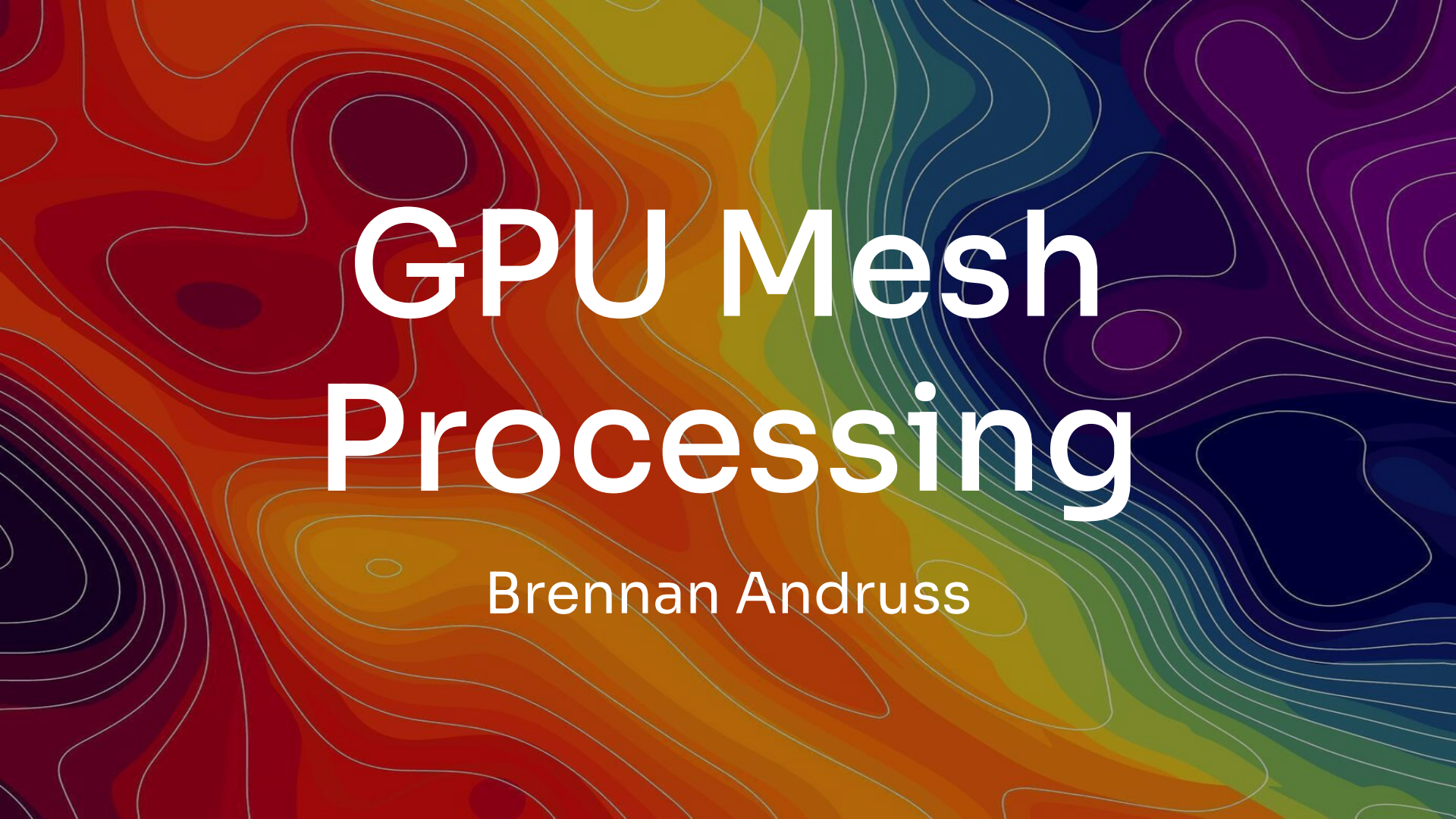




GPU Mesh Processing

Brennan Andruss

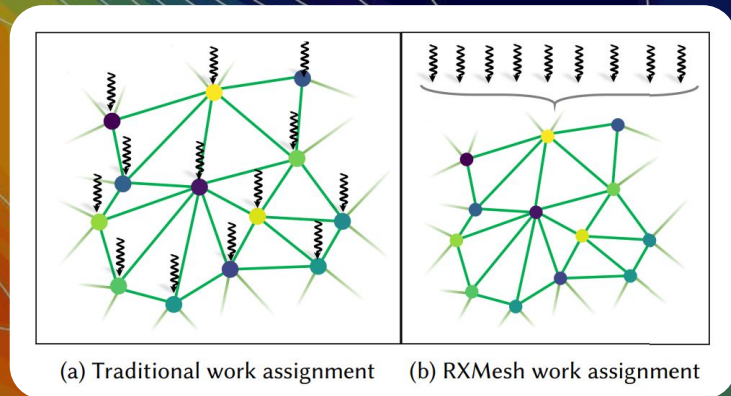
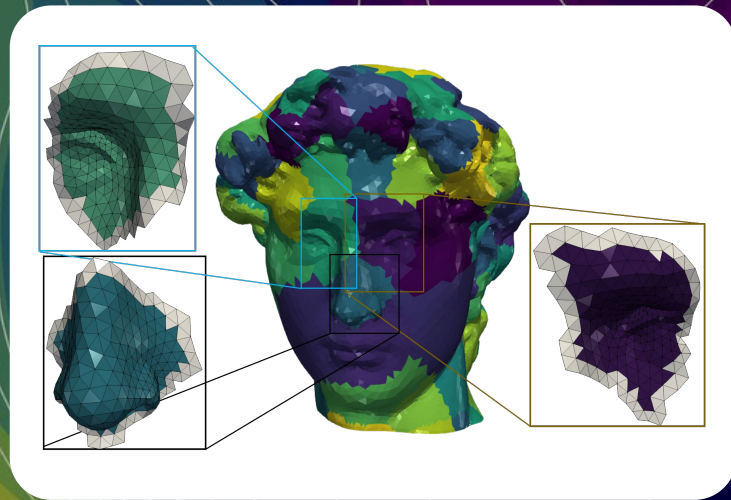
Review: RXMesh

GPU Mesh Data Structure

- Meshes patched for locality
- Queries for vertices, edges, and faces
 - Stored as connectivity matrices → computed on GPU
- Work mapping behind-the-scenes
 - Program queries for features → system optimizes for patches

RXMesh Library

- Authors developed GPU mesh processing library built on RXMesh



Using RXMesh

RXMesh Programming Model

- Create attributes for features
- Define a query kernel function
- Operate on features and update attributes
- Run query kernel on a RXMesh

```
// Vertex Coordinates
auto vertex_pos = *rx.get_input_vertex_coordinates();

// Face Normal
auto face_normal = *rx.add_face_attribute<float>("fNormals", 3);
```

```
rx.run_query_kernel<Op::FV, blockThreads>([=] __device__(FaceHandle face_id, VertexIterator &fv) mutable
{
    // Get the face's three vertex coordinates
    // Compute and normalize the face normal
    // Store the normals
    normals.from_glm(face_id, n);
```

RXMesh Sample Applications

RXMesh Static

- Mean Curvature Flow
- Geodesic Distance
- Bilateral Filtering

RXMesh Dynamic

- Isotropic Remeshing
- Surface Tracking
- Delaunay Edge Flip
- Shortest Edge Collapse

Face and Vertex Normals

Method

- Run FV query kernel with a face handle and vertex iterator
 - Face Handle → Get ID of face
 - Vertex Iterator → Get vertices of face
- Pass vertex coords as a vertex attribute
 - Get coordinates in kernel using vertex iterator
- Compute normal with cross product
- Store / accumulate normal in normals attribute
 - Vertex Normals: Normals from adjacent faces weighted by edge lengths

Geometry Processing: The Laplacian

The Laplacian (Δ)

- How much a function at a point differs from its neighbors
 - Negative value $\rightarrow$ higher, positive value $\rightarrow$ lower
- Continuous Function: $\Delta f = \text{div}(\nabla f) = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2 + \partial^2 f / \partial z^2$
- Discrete Function: $\Delta f(i) = \sum_{j \in N(i)} w_{ij} (f_j - f_i)$
 - “Average value of neighbors - value”
- Different forms of the Laplacian use different weights
 - Uniform Laplacian: Uniform weights $\rightarrow$ every neighbor contributes equally
 - Cotangent Laplacian: Uses angles and vertex positions $\rightarrow$ geometry-aware

Heat Diffusion

Heat Equation on a Surface

- Continuous Function: $\partial u / \partial t = \Delta u$
- Discrete Function: $\Delta u_i = 1/\deg(i) \sum_{j \in N(i)} u_j - u_i$

Euler's Method

- Compute heat diffusion over time by integrating over heat equation in discrete time steps
- Heat Equation: $u_i^{t+1} = u_i^t + dt \Delta u_i^t$

Heat Diffusion

Method

- Solve the heat equation in discrete steps for a number of iterations
- Define two vertex attributes for heat for double buffering
 - Old heat values needed to compute new values → don't overwrite values!
 - After each step, swap input and output buffers
- Run VV query kernel to compute heat diffusion at each step
 - Solve heat equation at each vertex with Euler's method
 - Read values from heatOld and write into heatNew
- Map normalized heat values to vertex colors with forEach kernel



Thank
You!